

A new heuristic based on a hypergraph representation for the tool switching problem

Housni Djellab*, Khaled Djellab, Michel Gourgand

Laboratoire d'Informatique (LIMOS), Université Clermont Ferrand II, 63177 Aubière Cedex, France

Abstract

This paper introduces a new approach for the tool switching problem arising in flexible manufacturing systems. We formulate the problem using a particular hypergraph representation and we propose an efficient heuristic to solve it. The performance of the heuristic is compared with the heuristics developed by Crama et al. (The International Journal of Flexible Manufacturing Systems 6 (1994) 33–54). The results show that our heuristic performs well in terms of computational efficiency and solution quality. © 2000 Elsevier Science B.V. All rights reserved.

Keywords: Tool switching; Manufacturing systems; Hypergraph; Heuristic

1. Introduction

In flexible manufacturing systems, numerical controlled (NC) machines are equipped with an automatic tool interchange device and used to manufacture different types of parts. We consider the problem of sequencing N parts of different types on a single NC-machine having a limited capacity Cap of the tool magazine. Each part type requires a subset of tools for its processing and the total number of tools is M ($M \geq Cap$). As the number of tools in the tool magazine cannot exceed Cap , some tools should be removed from the magazine in order to make room for tools needed for the next part type. The problem is to find the best part-type sequence such that the number of tool switches is

minimized. This problem has been addressed by Tang and Denardo [1] and Crama et al. [2]. The optimization of this kind of problem is useful when the time needed to change a tool is significant or the capacity of the tool transportation system is limited. For more details and other applications see references in [2,3].

The tool switching problem is defined by two interdependent problems, namely:

- Sequencing: Determine an optimal part-type sequence.
- Tooling: For the given part-type sequence, determine a tool loading sequence which minimizes the number of tool switches.

If the first problem is solved, then the second one can be done optimally using the “Keep Tool Needed Soonest” (KTNS) policy established by Tang and Denardo [1]. This policy has the following properties:

*Corresponding author. Address: Ecole des Mines de Nantes, Département Informatique, BP 20722, 44307 Nantes Cedex 03, France

1. At any instant, load all the tools that are required for the current part type.
2. If one or more tools are to be unloaded, the tools that are kept are those needed the soonest.

The correctness of the KTNS policy was first proved by Tang and Denardo [1]. Crama et al. [2] have given a compact proof, using a particular integer programming formulation, which is solved using a greedy algorithm. The KTNS policy was extended by Privault and Finke [4], using the network flow formulation in order to take into account the “non-uniform setups times”.

Many heuristics have been proposed for the tool switching problem which is proved to be NP-complete by Crama et al. [2]. These heuristics have been classified by Crama et al. [2] as follows:

Construction strategies: Construct job sequence step by step by exploiting a particular structure of the tool switching problem. These heuristics can be classified into two categories: In the first one, the tool switching problem is formulated as traveling salesman problem, where the distance between two jobs is an estimation of the number of tool switches required between these jobs (see [1,2,5]). The second category was sparked by Crama et al. [2] and Privault [5] who propose a “greedy” heuristic that successively adds the jobs to a current subsequence according to some priority criteria.

Improvement heuristics: They iteratively improve a starting job sequence. Several local search heuristics (2-exchange, tabu search, ...) for the tool switching problem have been tested in [1,2,6–9].

This paper is concerned with the tool switching problem and has two main purposes. First, to formulate the tool switching problem using a particular hypergraph representation. Second, to propose an efficient procedure to solve the problem.

The paper is structured as follows. In Section 1, we provide a general formulation of the hypergraph representation problem. We describe in Section 2 the proposed heuristic and its variants for solving the hypergraph representation problem. Section 3 is devoted to the illustration of how the proposed approach can be used to solve the tool switching problem. In Section 4, the computational performance of our heuristic is compared with the heuristics

developed by Crama et al. [2]. Section 5 is a brief conclusion.

2. Hypergraph and its representation

This section introduces definitions and notations for the hypergraph representation.

Definition 1.1. (Berge [10]): A hypergraph $\mathcal{G} = (X, E)$ consists of a finite set X of vertices $x_1, \dots, x_n$ and a family E of nonempty edges $e_i (i = 1, \dots, m)$ satisfying $\bigcup_{i=1}^m e_i = X$.

A hypergraph is represented by an edge-vertex incidence matrix B where $b_{ij} = 1$ if $x_j \in e_i$ and 0 otherwise (see Fig. 1). In such a matrix, each row corresponds to an edge and each column corresponds to a vertex.

Let $\mathcal{F} = (X, E, C, <)$ be a hypergraph, where X is the set of vertices submitted to a partial order $<$ and $C = (C_1, \dots, C_m)$, C_i is a cost associated to edge $e_i \in E$. Let $E(x)$ be a set of edges which contains x ($|E(x)|$ denotes the degree of vertex x).

We define an injective projection of the pair $(X, <)$ as any permutation $\pi: X \rightarrow I_n = \{1, \dots, n\}$ satisfying the property:

$$\forall x, y \in X, x < y \Rightarrow \pi(x) < \pi(y).$$

We define $l_\pi(A)$ as the length of a subset A ($A \subset X$) according to permutation π is as follows:

$$l_\pi(A) = \text{Max}_{x \in A} \pi(x) - \text{Min}_{x \in A} \pi(x).$$

The subset A of X is a π -interval if the following condition holds: $l_\pi(A) = |A| - 1$, where $|A|$ is the cardinal of A . For a subset A of X we note by $t_\pi(A)$ the number of π -intervals in A .

Example 1.1. Consider the hypergraph shown in Fig. 1. The partial order on vertices is defined by the constraints: $x_1 < x_2$, $x_1 < x_6$, $x_6 < x_2$, $x_6 < x_5$ and $x_6 < x_8$. The cost of each edge is assumed to be 1.

The permutation $\pi_1 = (x_3, x_4, x_5, x_2, x_1, x_6, x_7, x_8)$ cannot be an injective projection of $(X, <)$ because constraint $x_1 < x_2$ is not satisfied. With the injective projection $\pi_2 = (x_3, x_1, x_6, x_2, x_5, x_7, x_4, x_8)$ (see Fig. 2) the edge $e_1 = \{x_1, x_4, x_5, x_6\}$ is of

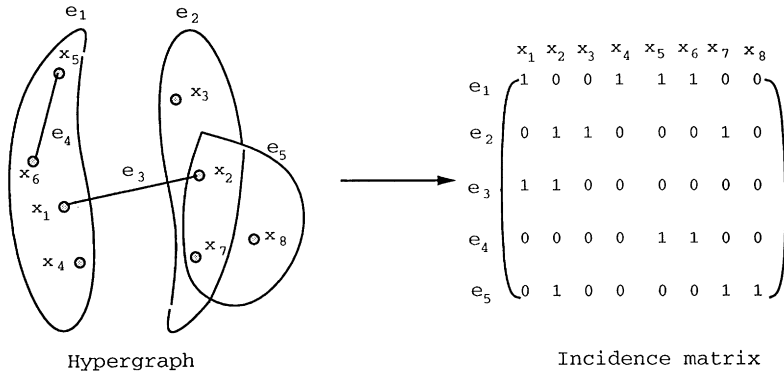
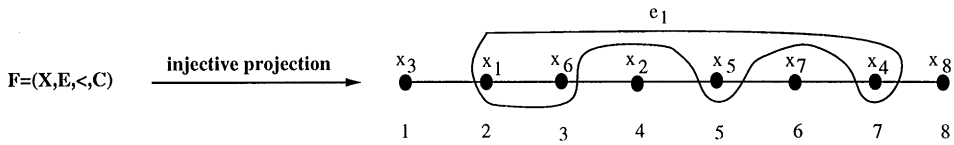


Fig. 1. A hypergraph and its corresponding incidence matrix.

Fig. 2. The representation of the edge e_1 in the π_2 injective projection (e_1 -projection).

length $l_\pi(e_1) = 5$. It is not a π -interval, as there are only three π -intervals, $e_1^1 = \{x_1, x_6\}$, $e_1^2 = \{x_5\}$, $e_1^3 = \{x_4\}$, and therefore $t_\pi(A) = 3$.

A hypergraph representation means that we can make a projection of all edges according to π . With these notations let us define the following combinatorial problem: Minimizing the number of total weighted gap in edge-projection (MGEP).

Data. A hypergraph $\mathcal{F} = (X, E, C, <)$;

Problem (MGEP). Find an injective projection of π of $(X, <)$ which Minimizes the number of total weighted gap in edges-projection MGEP: $T_\pi(E) = \sum_{e \in E} C(e)t_\pi(e)$.

The MGEP problem is a weighted version of the matrix permutation problem (consecutive block minimisation problem, see Kou [11]).

Definition 1.2. We recall that $\mathcal{F} = (X, E, C, <)$ is an interval hypergraph if there exists an injective projection in which each edge is a π -interval (i.e. $t_\pi(e) = 1 \forall e \in E$).

It is easy to see that the MGEP problem is NP-complete. Indeed, the consecutive block minimization problem presented in [11] is the unweighted version of MGEP. When $\mathcal{F}$ is an interval hypergraph the associated problem MGEP is polynomially solvable by means of algorithms proposed by Booth and Lueker [12] and Baptiste et al. [13].

3. Heuristics for the MGEP problem

In this section we propose an iterative construction procedure, which consists of a number of stages.

- At the first stage, we exploit the structure of the problem in order to construct an initial sequence of vertices. This stage is based on the use of an insertion heuristic presented in Section 3.1
- At the stage i ($i > 1$), we use the current sequence to construct a better one. At this stage, we use an iterative heuristic procedure for a fixed number of iterations.

In the next sections we present these heuristics.

3.1. Best insertion heuristic (BI)

Let $S_i = \{x_1, \dots, x_i\}$ be the current subset of X and suppose that we have the corresponding injective projection of vertices $\pi_i = x_{\alpha[1]}, \dots, x_{\alpha[i]}$ compatible with the partial order $<$.

Let us consider now a vertex x in $X \setminus S_i$, and let us find its best position in π_i which is compatible with the partial order $<$ and minimize our objective $T_{\pi}(E)$. The feasible interval position of x is:

$$\underline{S}_i(x) = \begin{cases} \text{Max } \{\pi_i(s) | s < x \text{ and } s \in S_i\} & \text{if } \exists s \in S_i \text{ such that } s < x, \\ 0 & \text{otherwise,} \end{cases}$$

$$\overline{S}_i(x) = \begin{cases} \text{Min } \{\pi_i(s) | x < s \text{ and } s \in S_i\} & \text{if } \exists s \in S_i \text{ such that } x < s, \\ i + 1 & \text{otherwise.} \end{cases}$$

In order to respect the partial order, the vertex x must be in the interval $[\underline{S}_i(x) + 1, \overline{S}_i(x)]$. For each position j in this interval we calculate its cost and then we choose the best one, which corresponds to the minimum cost. Let $j \in [\underline{S}_i(x) + 1, \overline{S}_i(x)]$ and $S_j = \{x[1], \dots, x[j]\}$.

If vertex x is placed at position j , the cost generated is $C(j, x, S_i) = \sum_{e \in Z} C(e) t_{\pi_i}(e)$, where

$$Z = \bigcup_{s=1}^3 E_s(x, j),$$

$$E_1(x, j) = \{e \in E | x \in e, x_j, x_{j+1} \notin e, e \cap S_j \neq \emptyset\},$$

$$E_2(x, j) = \{e \in E | x \in e, x_j, x_{j+1} \notin e, e \cap (S_i \setminus S_j) \neq \emptyset\}$$

and

$$E_3(x, j) = \{e \in E | x \notin e, x_j, x_{j+1} \in e\}.$$

The best position of vertex x is j^* such that $C(j^*, x, S_i) = \text{Min } \{C(j, x, S_i) | j \in [\underline{S}_i(x) + 1, \overline{S}_i(x)]\}$. Once the vertex x is placed, we take another vertex not yet placed using the same technique until all vertices are placed.

The best insertion heuristic BI can be sketched as follows.

Heuristic BI

Input: A hypergraph $(X, E, C, <)$;
A priority order p (used for vertex insertion);

Step 1: $Y \leftarrow X$;

$\pi \leftarrow \emptyset$

Step 2: While $Y \neq \emptyset$ do

- let x be the vertex of Y which has the highest priority in p ;
 - introduce x in π at position j^* such that:
 $C(j^*, x, S) = \min \{C(j, x, S) | j \in [\underline{S}(x) + 1, \overline{S}(x)]\}$;
 - $Y \leftarrow Y \setminus \{x\}$;
 - $S \leftarrow S \cup \{x\}$;
- endofwhile;*

Output: π_p an injective projection of $(X, <)$.

BI runs in time $O(mn^2)$. Indeed, at iteration i , ($i = 1, \dots, n$) there are i possible positions. The complexity of the cost evaluation at each of these position is $O(m)$.

Obviously, the performance of BI depends on the choice of the priority order p used for vertices insertion. In the next section, we give an iterative procedure of BI where the priority order is changed at each iteration by taking into account the current injective projection of vertices.

3.2. Iterative best insertion procedure (IBI)

One of the drawbacks of the heuristic BI and in general the construction heuristic is that it cannot assure optimality. In order to palliate this difficulty, we propose some techniques to “Iterate” the BI heuristic.

For the MGEP problem we wish to determine the optimal order (permutation) of vertices. Our procedure is based on the following rule: after each call of the BI heuristic we add a precedence constraint between two incomparable vertices x and y , and we transitively close over $(X, <)$. We continue until a total order between vertices is obtained. The efficiency of the IBI heuristic depends on the choice of the constraint we add after each call of the BI heuristic. We propose in Sections 3.2.1 and 3.2.2 two criteria for this choice.

Let us define the meaning of the priority order in the studied problem. A priority order on vertices in X is a one-to-one function $p : X \rightarrow \{1, \dots, n\}$, where n is the number of vertices in X . We call $p(x)$ the priority level of vertex x . We define p' the inverse of the priority order p , $p' : X \rightarrow \{1, \dots, n\}$, where

$p'(x) = n + 1 - p(x)$. Note that for any vertices x and y in X we have: $p(x) < p(y) \Leftrightarrow p'(y) < p'(x)$. Given two vertices x and y . There are two cases:

- vertex x has a higher priority level than y ,
- vertex x has a lower priority level than y .

In order to take into account those two cases we use a priority order and its inverse at each step of our heuristic.

The choice of the incomparable vertices is made with one of the criteria described below.

3.2.1. Criterion I

Let p be a priority order of vertices and p' the inverse of p . We choose a couple of incomparable vertices x and y , which verifies:

- (i) $\pi_p(x) < \pi_p(y)$,
- (ii) $\pi_{p'}(x) < \pi_{p'}(y)$,
- (iii) $|p(x) - p(y)|$ is maximal for the properties (i) and (ii).

If this couple of vertices exists, we give the following signification: Without any restriction, suppose that x has a higher priority level than y in p . Accordingly, y has a higher priority level than x in p' . From priority orders p and p' , where $p(x) > p(y)$ and $p'(y) > p'(x)$, we have placed x before y in both π_p and $\pi_{p'}$. In other words, the related priority level of vertices x and y do not prevent x from placing before y (properties (i) and (ii)). We choose the couple (x, y) such as the difference between priorities of x and y is maximal. We add the constraint $(x < y)$ and its transitive closure in $(X, <)$.

If such a couple of vertices x and y exists in each iteration, the convergence is assured and there are at most $n(n - 1)/2$ iterations (maximum number of arcs in a digraph). This kind of couples do not always exist. Therefore, we propose another criterion in order to make sure that the IBI procedure converges. In fact, if criterion I is not verified, the following criterion is always verified.

3.2.2. Criterion II

For reason of symmetry we suppose that $T_{\pi_p}(E)$ is smaller than $T_{\pi_{p'}}(E)$. Here, we replace the properties (ii) and (iii) by:

- (ii)' $\pi_{p'}(x) > \pi_{p'}(y)$,
- (iii)' $|\pi_{p'}(x) - \pi_{p'}(y)|$ is minimal for properties (i) and (ii)'.

In the injective projection $\pi_{p'}$, we have placed y before x (property (ii)'). We add the constraint $x < y$ which goes against property (ii)'. As there are several couples which verify this property we choose the couple (x, y) which minimizes the difference between the positions of x and y in $\pi_{p'}$. We add a constraint $(x < y)$ and its transitive closure in $(X, <)$.

We iterate this process by applying criteria I and II until a total order between vertices is obtained. Combining the Best Insertion heuristic and the criteria we obtain the Iterative Best Insertion procedure is (IBI) sketched as follows.

Procedure IBI

Input: A hypergraph $(X, E, C, <)$;

- Start the priority order p in the decreasing degree of vertices;
- Start the priority p' in the increasing degree of vertices;

Step 1: $(X, <) \leftarrow (X, <); \pi_{p^*} \leftarrow p$;

Step 2: While $(X, <)$ is not a total order do

- BI $((X, E, C, <), p, \pi_p)$;
 - BI $((X, E, C, <), p', \pi_{p'})$;
 - if $T_{\pi_{p'}}(E) \leq T_{\pi_{p^*}}(E)$ then $\pi_{p^*} = \pi_{p'}$;
 - if $T_{\pi_p}(E) \leq T_{\pi_{p^*}}(E)$ then $\pi_{p^*} = \pi_p$;
 - find an incomparable couple (x, y) verifying criterion I otherwise criterion II;
 - $(X, <) \leftarrow (X, <) \cup (x < y)$;
 - the priority p is randomly generated,
 $p = (x_1, \dots, x_n)$;
 - $p' \leftarrow (x_n, \dots, x_1)$;
- endofwhile;*

Output: π_{p^*} permutation.

IBI runs in time $O(mn^4)$. Indeed, it requires $n(n - 1)/2$ calls of BI heuristic in step 2 each of which has complexity $O(mn^2)$. If we add at each iteration in step 2, $k\%$ of $n(n - 1)/2$ constraints¹ by criterion I completed by criterion II, then the complexity of IBI and BI are equivalent (we suppose that k is independent of the problem size). Indeed, the number of iterations in step 2 will be a constant: $100/k$. This value does not change the quality of

¹ We add $[kn(n - 1)/(2 \times 100)] + 1$ constraints, where $[x]$ is the integer value of x .

solutions and significantly reduces the CPU time (see Section 5.2).

We compared the efficiency of our procedures with an exhaustive enumeration method proposed in [14]. The results obtained encourage us to use this procedure for tool switching problem. As we will show in the next section the tool switching problem can easily be translated to the MGEP problem. Therefore the IBI procedure can be used to solve it.

Example 2.3. Let us consider the hypergraph $\mathcal{F} = (X, E, C, <)$ where $X = \{1, 2, \dots, 8\}$ is the set of vertices, $E = \{e_1, e_2, \dots, e_{15}\}$ is the set of edges defined by:

$$e_1 = \{1, 6, 7\}, C_1 = 5,$$

$$e_2 = \{1, 3, 5\}, C_2 = 6,$$

$$e_3 = \{1, 3, 6\}, C_3 = 6,$$

$$e_4 = \{2, 3, 5, 7, 8\}, C_4 = 4,$$

$$e_5 = \{3, 5\}, C_5 = 1,$$

$$e_6 = \{3, 6\}, C_6 = 3,$$

$$e_7 = \{2, 4, 6, 7\}, C_7 = 2,$$

$$e_8 = \{3, 4\}, C_8 = 4,$$

$$e_9 = \{2, 7\}, C_9 = 3,$$

$$e_{10} = \{1, 4, 5, 6, 8\}, C_{10} = 5,$$

$$e_{11} = \{6, 8\}, C_{11} = 5,$$

$$e_{12} = \{2, 4\}, C_{12} = 4,$$

$$e_{13} = \{1, 2, 7, 8\}, C_{13} = 4,$$

$$e_{14} = \{1, 2, 4, 5, 7, 8\}, C_{14} = 3,$$

$$e_{15} = \{4, 5, 8\}, C_{15} = 4.$$

A partial order $<$ on vertices is defined by constraints: $(1 < 6, 2 < 5, 4 < 1 \text{ and } 7 < 8)$.

The projection $\pi_1 = (7, 2, 4, 8, 5, 1, 6, 3)$ found by the procedure IBI with the initial priority order $p = (1, 2, 3, 4, 5, 6, 7, 8)$ is a solution of the problem MGEP. The associated cost is $T_{\pi_1}(E) = 39$, the number of gaps (0-blocks underlined) for this solution is 10 (see Table 1).

4. The tool switching problem

We consider the problem of sequencing N jobs on a single NC-machine having a limited capacity

Table 1

Solution of problem $n = 8, m = 15$

Place	1	2	3	4	5	6	7	8	Number
Vertices: 7	2	4	8	5	1	6	3		of gaps
Edges									
1	1	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	1	1	0	1
2	0	0	0	0	1	1	<u>0</u>	1	1
3	0	0	0	0	0	1	1	1	0
4	1	1	<u>0</u>	1	1	<u>0</u>	<u>0</u>	1	2
5	0	0	0	0	1	<u>0</u>	<u>0</u>	1	1
6	0	0	0	0	0	0	1	1	0
7	1	1	1	<u>0</u>	<u>0</u>	<u>0</u>	1	0	1
8	0	0	1	<u>0</u>	<u>0</u>	<u>0</u>	<u>0</u>	1	1
9	1	1	0	0	0	0	0	0	0
10	0	0	1	1	1	1	1	0	0
11	0	0	0	1	<u>0</u>	<u>0</u>	1	0	1
12	0	1	1	0	0	0	0	0	0
13	1	1	<u>0</u>	1	<u>0</u>	1	0	0	2
14	1	1	1	1	1	1	0	0	0
15	0	0	1	1	1	0	0	0	0

Cap of the tool magazine. Each job requires a subset of tools for its processing and the total number of tools is M . If these tools are not present in the magazine, one or more tools must be replaced by others. The problem is to find the best job sequence such that the number of tool switches is minimized [1,2]. This problem can be formulated in terms of the MGEP problem. The hypergraph $\mathcal{F} = (X, E, C, <)$ is defined by:

- the set of jobs to be processed (X);
- the set of jobs which requires the tool i (edge e_i in E);
- the partial order between jobs $<$ which can be empty;
- the cost of switching tool i ($C(e_i), e_i \in E$).

The MGEP problem with capacity Cap is defined as follows:

Data. A hypergraph $\mathcal{F} = (X, E, C, <)$; a capacity Cap;

Problem (MGEP-Cap). Find an injective projection π of $(X, <)$ and a super edge $E' = \{e'_1, \dots, e'_m\}$ containing $E = \{e_1, \dots, e_m\}$ (i.e. $e_i \subseteq e'_i, i = 1, \dots, m$) such that every vertex has degree Cap and $T_\pi(E') \sum_{e' \in E'} C(e') t_\pi(e')$ is minimized. Note that $C(e')$ and $C(e)$ have the same value.

We show that the resolution of problem MGEP-Cap gives an optimal solution for the tool switching problem. For the non-uniform setups times we use the method (network flow formulation) proposed by Privault et al. [4]. The adaptation of the procedure IBI to the tool switching problem is as follows.

Procedure IBI-Tools

Input: A hypergraph $(X, E, C, <)$; A capacity Cap;
Start the priority order p with jobs requiring the most tool;

Start the priority p' with jobs requiring less tool;

Step 1: $(X, <)(X, <); E_s \leftarrow E$ (E_s is a temporaire variable); $\pi_{p^*} \leftarrow p$

Step 2: While $(X, <)$ is not a total order do

- BI $((X, E, C, <), p, \pi_p)$;
 - $T_{\pi_p}(E_s) \leftarrow \text{KTNS}(E_s, \pi_p, \text{Cap})$; $E_s \leftarrow E$;
 - BI $((X, E, C, <), p', \pi_{p'})$;
 - $T_{\pi_{p'}}(E_s) \leftarrow \text{KTNS}(E_s, \pi_{p'}, \text{Cap})$; $E_s \leftarrow E$;
 - if $T_{\pi_p}(E_s) \leq \text{cost}(\pi_{p^*})$ then $\pi_{p^*} \leftarrow \pi_p$;
 - if $T_{\pi_{p'}}(E_s) \leq T_{\pi_{p^*}}(E_s)$ then $\pi_{p^*} \leftarrow \pi_{p'}$;
 - find an incomparable couple (x, y) verifying criterion I otherwise criterion II;
 - $(X, <) \leftarrow (X, <) \cup (x < y)$;
 - the priority $p = (x_1, \dots, x_n)$ is randomly generated;
 - $p' \leftarrow (x_n, \dots, x_1)$;
- endofwhile;

Output: π_{p^*} sequence of jobs.

We recall here the KTNS policy used in the procedure IBI. The moment in time after processing the n th job, but before any tools are switched is called instant n . The KTNS policy has the following properties:

1. At any instant, load all the tools that are required for the current job.
2. If one or more tools are to be unloaded, the tools that are kept are those needed the soonest. The KTNS policy output is the cost of permutation in the hypergraph defined by E_s and capacity Cap.

Example 3.1. In order to illustrate the tool switching problem, consider the input data considered by

Table 2
Tools-jobs incidence matrix

Jobs: Tools	1	2	3	4	5	6	7	8	9	10
1	1	1	0	0	0	0	1	0	0	1
2	0	0	1	0	0	0	0	0	0	1
3	0	1	0	0	0	1	0	1	0	0
4	1	0	0	0	0	0	0	0	0	1
5	0	1	0	0	0	0	1	1	1	0
6	0	0	1	0	1	0	0	0	0	0
7	0	0	1	1	0	0	1	0	1	0
8	1	0	1	0	0	0	0	1	0	0
9	1	0	0	0	0	0	1	0	0	0

Table 3
Solution of π_2 sequence

Place Jobs: Tools	1 4	2 9	3 7	4 2	5 8	6 6	7 1	8 10	9 5	10 3	Number of tool setups
1	1	1	1	1	1	1	1	1	0	0	1
2	0	0	0	0	0	0	0	1	1	1	1
3	0	0	0	1	1	1	0	0	0	0	1
4	0	0	0	0	0	0	1	1	1	0	1
5	1	1	1	1	1	1	0	0	0	0	1
6	0	0	0	0	0	0	0	0	1	1	1
7	1	1	1	0	0	0	0	0	0	1	2
8	0	0	0	0	1	1	1	1	1	1	1
9	1	1	1	1	0	0	1	0	0	0	2

Tang et al. [1] and described in Table 2 where $n = 10$, $m = 9$, and Cap = 4.

If we start the priority order p with jobs requiring the most tools, we obtain the sequence $\pi_1 = (1, 5, 6, 9, 2, 8, 10, 3, 4, 7)$ using the heuristic BI. The corresponding tool setups is 16 after the application of KTNS policy. After 10 times of the procedure IBI-Tools, the best feasible sequence is $\pi_2 = (4, 9, 7, 2, 8, 6, 1, 10, 5, 3)$ yielding 11 tool setups (7 switches). A representation of this solution is given in Table 3.

5. Computational results

In order to evaluate our modeling approach and the proposed procedure we use 16 tests (m, n, Cap) described in [2] for the tool switching problem.

Table 4
Problems and capacities of the tool magazine

(m, n)	Cap ₁	Cap ₂	Cap ₃	Cap ₄
(10, 10)	4	5	6	7
(20, 15)	6	8	10	12
(40, 30)	15	17	20	25
(60, 40)	20	22	25	30

Each test contains 10 problems and for each type of tests, a generic set of data (see Table 4) is used.

The results presented are average values of the ten generated problems.

5.1. Evaluation of the subsequence

The Best Insertion heuristic used in this paper is not the same as that proposed in [8,5] for the tool switching problem. In our heuristic BI the evaluation of the subsequence is tested by the following two local criteria:

- The first one is the number of tool setups after applying the KTNS policy called “A-KTNS”.
- The second is the number of tool setups without applying KTNS policy called “W-KTNS”.

The results obtained by the second criterion are better than those obtained by the first one (see Fig. 3(a)). Furthermore, second criterion requires less computational time than the first one (see Fig. 3(b)). In fact, the complexity of BI is $O(mn^3)$ with A-KTNS; and $O(mn^2)$ with W-KTNS.

5.2. The influence of parameter k

Let us study the case when we add $k\%$ of $n(n-1)/2$ constraints by criterion I completed by criterion II. The different values of k used are, $k = 1$ (we add one constraint in each iteration), 5%, 10%, and 15%. As we have mentioned above, this percentage of constraints to be added at each iteration in the procedure IBI-Tools considerably reduces the CPU time (see Fig. 4) without changing the quality of solutions (see Fig. 5).

5.3. Number of constraints to be added by criteria I and II

In this section, we compare the number of constraints to be added with criteria I and II. Table 5a and b shows the following results:

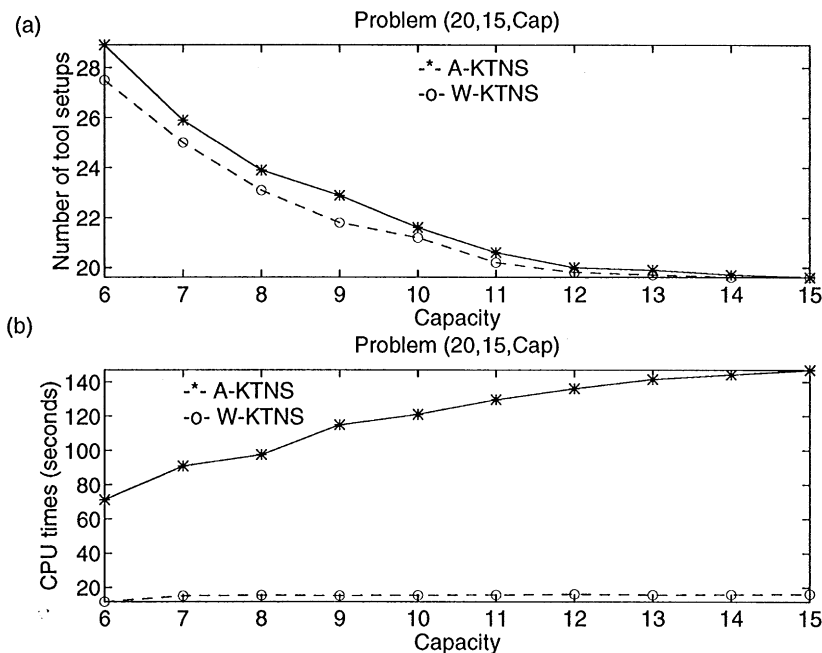


Fig. 3. Comparison of two local criteria in the IBI procedure.

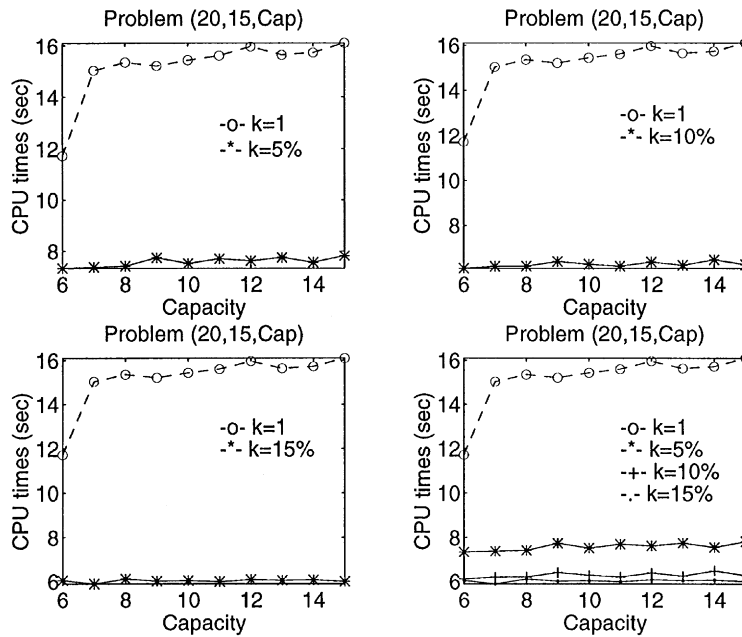


Fig. 4. Variation of total CPU time with percentage.

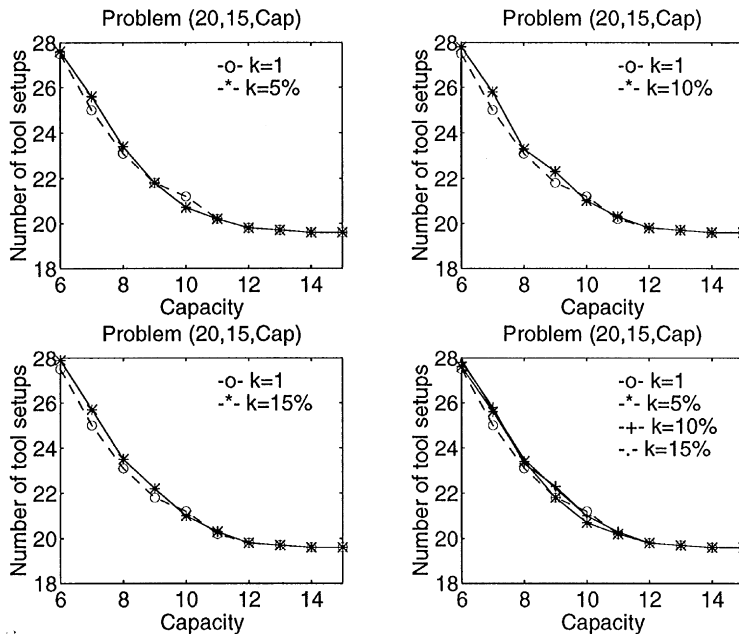


Fig. 5. Variation of the number of tool setups with percentage of total number of constraints to be added.

Table 5
Comparison between criteria I and II

(a) Problems	(10,10,4)	(10,10,5)	(10,10,6)	(10,10,7)	(20,15,6)	(20,15,8)	(20,15,10)	(20,15,12)
Criterion I	68.73	68.95	68.80	68.73	60.68	60.65	60.79	60.66
Criterion II	31.27	31.05	31.20	31.27	39.32	39.35	39.21	39.34
(b) Problems	(40,30,15)	(40,30,17)	(40,30,20)	(40,30,25)	(60,40,20)	(60,40,22)	(60,40,25)	(60,40,30)
Criterion I	49.26	49.17	49.26	49.29	45.59	45.57	45.64	45.61
Criterion II	50.74	50.83	50.74	50.71	54.41	54.43	54.36	54.39

Table 6
Number of tool setups

Complexity problems ($m.n$.Cap)	$O(m.n^4)$ IBI-tools ^a $A = 1, k = 1$	$O(m.n^2)$ IBI-tools $A = 1, k = 5\%$	$O(m.n^3)$ Farthest insertion	$O(m.n^4)$ FI block Mini	$O(m.n)$ Interval	$O(m.n^3)$ Load-and- optimize
(10,10,4)	13.3	13.6	14.8	15.1	16.2	13.9
(10,10,5)	11.4	11.7	13.3	12.7	12.8	12.3
(10,10,6)	10.3	10.4	12.1	11.3	11.3	10.9
(10,10,7)	10.1	10.1	11.3	10.5	10.4	10.4
(20,15,6)	27.5	27.6	30.6	29.7	33.3	29.8
(20,15,8)	23.1	23.4	29.7	24.6	26.1	26.7
(20,15,10)	21.2	20.7	26.1	21.7	22.1	25.1
(20,15,12)	19.8 ^b	19.8	22.6	20.1	20.3	22.8
(40,30,15)	119.6	120.3	124.3	123.6	148.2	132.5
(40,30,17)	99.4	102.9	111.5	104.9	125.8	122.3
(40,30,20)	83.4	84.5	102.8	86.1	100.1	103.8
(40,30,25)	62.7	61.8	79.2	65.3	72.6	78.3
(60,40,20)	220.3	222.1	226.2	232.1	276.5	253.9
(60,40,22)	202.5	205.1	208.5	206.2	245.3	237.9
(60,40,25)	170.1	176.3	195.5	177.3	208.9	217.5
(60,40,30)	137.8	140	174.2	144.1	164.1	174.2

^a A is the iterations number of the IBI-tools procedure.

^b The value 19.8 is smaller than $m = 20$ because there is a null row in the corresponding matrix.

- for small problems (see Table 5a), the number of constraints generated by criterion I is larger than those generated by criterion II,
- for large size problems (see Table 5b), the inverse relationship holds.

From the computational results we have analyzed criteria I and II. In order to illustrate this analysis D_1 (resp. D_2) denote the set of iteration numbers where constraints have been added by criterion

I (resp. criterion II). More than 99% of the D_1 elements have iteration number smaller than those in D_2 . In other words, the transition from an empty partial order to a total order consists in a sequence of constraints added by criterion I, followed by another sequence added by criterion II. This observation is valid for all problems of this paper. Besides, the comparison of the number n_1 of elements in D_1 with the number n_2 of elements in D_2 strongly depends on the size of the problem. In

Table 7
Number of tool setups

Complexity problems (m, n, Cap)	$O(m, n^4)$ IBI-tools $A = 50, k = 1$	$O(m, n^2)$ IBI-tools $A = 50, k = 5\%$	$O(m, n^3)$ Simple greedy	$O(m, n^4)$ Multi-start greedy	$O(m, n^2)$ Restricted 2-OPT	Global 2-OPT
(10,10,4)	13.1	13.2	14.8	13.8	16.6	14.3
(10,10,5)	11.2	11.2	12.1	11.6	13.9	12.1
(10,10,6)	10.3	10.3	10.9	10.6	12.2	10.6
(10,10,7)	10.1	10.1	10.2	10.1	11.1	10.1
(20,15,6)	26.4	26.6	30.1	27.7	35.4	29.1
(20,15,8)	21.4	21.7	23.9	22.6	29.3	23.7
(20,15,10)	20.0	20.1	21.1	20.3	24.8	21.3
(20,15,12)	19.6	19.6	19.8	19.6	22.3	19.8
(40,30,15)	116.8	118.4	126.5	120.6	148.1	120.4
(40,30,17)	97.8	99.4	105.3	101.2	125.6	100.2
(40,30,20)	78.1	78.4	84.3	79.3	102.2	81.4
(40,30,25)	57.9	58.1	61.5	58.3	76.4	60.2
(60,40,20)	218.1	222.6	233.2	223.9	266.2	221.7
(60,40,22)	192.2	195.7	204.7	199.9	237.8	196.7
(60,40,25)	163.1	164.1	173.6	164.9	208.2	163.8
(60,40,30)	128.7	129.7	135.7	129.6	165.7	133.1

fact, whenever the size of the problem increases, n_1 decreases and n_2 increases. For instance, in Table 5 shows that for an average of 40 problems (10, 10), the ratio is $n_1/n_2 \simeq \frac{69}{31}$. This ratio decreases to $\frac{45}{55}$ for the problems (60, 40).

In order to explain that the value n_1/n_2 decreases denote by $x < y$ the constraint added at an iteration i . This implies that $\text{nb}(i)$ constraints are added because of “transitive closure”. The details of the numerical results enable us to study the curve of $\text{nb}(i)$ for different iteration numbers. In the first iterations, the constraints $x < y$ are added, mainly using criterion I. If the problem size is small the first constraints added are likely to be adjacent. In this case, the number $\text{nb}(i)$ of added constraints by “transitive closure” is significant. On the other hand, in the case of large size problems, the value of $\text{nb}(i)$ is less significant. In conclusion, the number $\text{nb}(i)$ is conversely proportional to the problem size during the first iterations.

5.4. Comparison of methods

Tables 6 and 7 list performance results of our procedures and those developed by Crama over 16

tests problems. The results for each heuristic are given in the columns. The following observations can be made:

- the solutions produced by our procedures are better than Crama’s;
- as we have seen in Figs. 4 and 5 the percentage of constraints to be added at each iteration does not change the quality of solutions for any of the tested problems;
- the performance of our procedures is independent of the sparse or dense problems. We call sparse problems those for which the ratio max/Cap is small, and dense problems those for which this ratio is close to 1.

6. Conclusion

We have analyzed the tool switching problem and shown that the hypergraph representation is well suited for solving the problem. The hypergraph representation we have used is called MGEP-Cap. We have presented a heuristic procedure for finding an equilibrium (local optimum)

to the MGEP-Cap and therefore to the tool switching problem. First, a heuristic BI has been proposed in order to determine an initial permutation of vertices, which exploits the special structure of the problem. We have iterated this heuristic by exploiting knowledge about the current permutation and we have obtained the procedure IBI.

The objective of our approach is to control the efficiency of the algorithms time and the quality of the solution.

- For large sized problems, if execution time is favored, we choose an appropriate percentage of constraints to be added. The convergence speed of our method is inversely proportional to the value of the parameter k . Fortunately, numerical results show that we can obtain a fast method (by increasing the value of k) without degrading significantly the quality of the solution.
- On the other hand, for small and medium sized problems one may privilege the quality of the solution. In that case, the choice of a constraint ($x < y$) to be added should be made attentively. One can use many priorities orders $(p_1, p'_1), \dots, (p_s, p'_s)$ in order to choose the constraint ($x < y$). We choose the constraint ($x < y$) which has the maximum voice “referendum” between $(p_1, p'_1), \dots, (p_s, p'_s)$.

Our research can be extended in two directions. The proposed heuristic could be implemented on parallel machines, when we use more than one priority order. Yet, another research avenue to apply our procedure is to solve other permutation problems.

It has been shown by computer simulations that the proposed algorithm gives good results. The suitability for the parallel implementation is another advantage of the proposed procedure.

References

- [1] C.S. Tang, E.V. Denardo, Models arising from flexible manufacturing machine, minimization of the number of tool switches Operations Research 36 (5) (1988) 767–784.
- [2] Y. Crama, A.W.J. Kolen, A.G. Oerlemans, F.C.R. Spieksma, Minimizing the number of tool switches on a flexible machine, The International Journal of Flexible Manufacturing Systems 6 (1994) 33–54.
- [3] Y. Crama, J. van de Klundert, The approximability of tool management problems, Maastricht Research School of Economics of Technology and Organizations METEOR, 1996.
- [4] C. Privault, G. Finke, Tool management on NC-machines, Proceeding of the International Conference on Industrial Engineering and Production Management, IEPM'93, Mons, Belgium, 1993, 667–676.
- [5] C. Privault, Modèles mathématiques pour la gestion offline et on-line des changements d'outils sur une machine flexible, Thesis de l'Université Joseph Fourier-Grenoble I, 1994.
- [6] J. Bard, A heuristic for minimizing the number of tool switches on a flexible machine, IIE Transaction 20 (4) (1988) 382–391.
- [7] A. Hertz, M. Widmer, La méthode TABOU appliquée aux problèmes d'ordonnancement, Automatique Production Informatique Industrielle RAIRO-APII 29 (4-5) (1995) 353–378.
- [8] J.-P. Follonier, Minimization of the number of tool switches on a flexible manufacturing machine, Belgian Journal of Operations Research, Statistics and Computer Science 34 (1) (1994) 55–72.
- [9] H. Djellab, K. Djellab, M. Gourgand, Hypergraph representation for the tool switching problem, Proceedings of the International Conference on Industrial Engineering and Production Management, IEPM'97, Lyon, France, 1997, pp. 342–352.
- [10] C. Berge, Hypergraphes: Combinatoire des ensembles finis, Gauthier-Villars, Paris, 1991.
- [11] L.T. Kou, Polynomial complete consecutive information retrieval problems, SIAM Journal on Computing 6 (1) (1977) 67–75.
- [12] K. Booth, G. Lueker, Testing for the consecutive one's property, interval graphs and planar graphs using PQ-Trees, Journal of Computers and System Science 13 (1976) 335–379.
- [13] P. Baptiste, C.H. Cho, J. Favel, M. Zouhri, Une caractéristique analytique des ordonnancements admissibles sous contraintes hétérogènes en flow shop, Automatique Production Informatique Industrielle RAIRO-APII 25 (1991) 87–102.
- [14] H. Djellab, Optimisation combinatoire dans les systèmes de production: Hypergraphes et méthodes d'amélioration itératives, Ph.D. Thesis de l'Université Blaise Pascal, Clermont-Ferrand II, 1997.